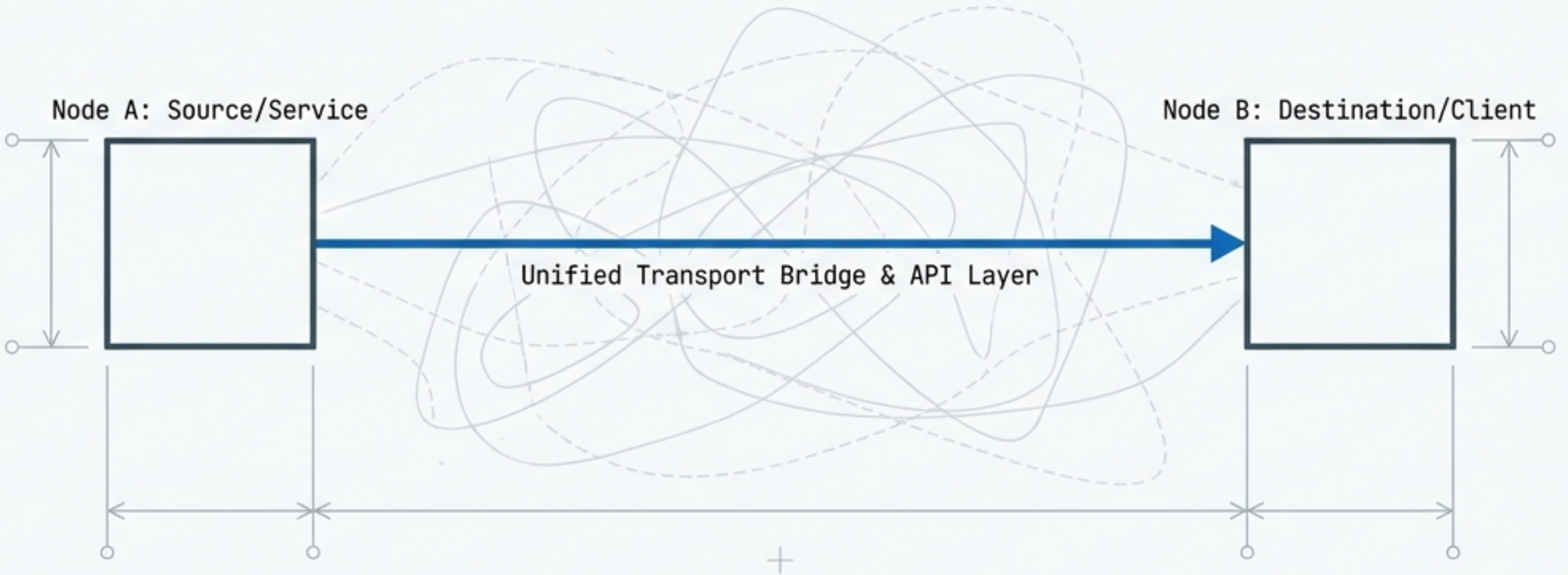
Unified Service Client Architecture (v0.32.5)

Centralising transport, eliminating wrappers, and unifying IN_MEMORY and REMOTE modes.



Affected Projects:

- osbot_fast_api
- mgraph_ai_service_cache & _client
- mgraph_ai_service_html_graph & _client

Date: January 2025
Status: Proposed

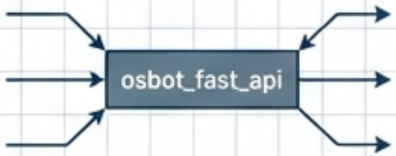
THE VISION: SEAMLESS MODE SWITCHING, ZERO CODE DUPLICATION.

GOAL:

Enable switching between **IN_MEMORY (TestClient)** and **REMOTE (HTTP)** modes without changing a single line of client code.

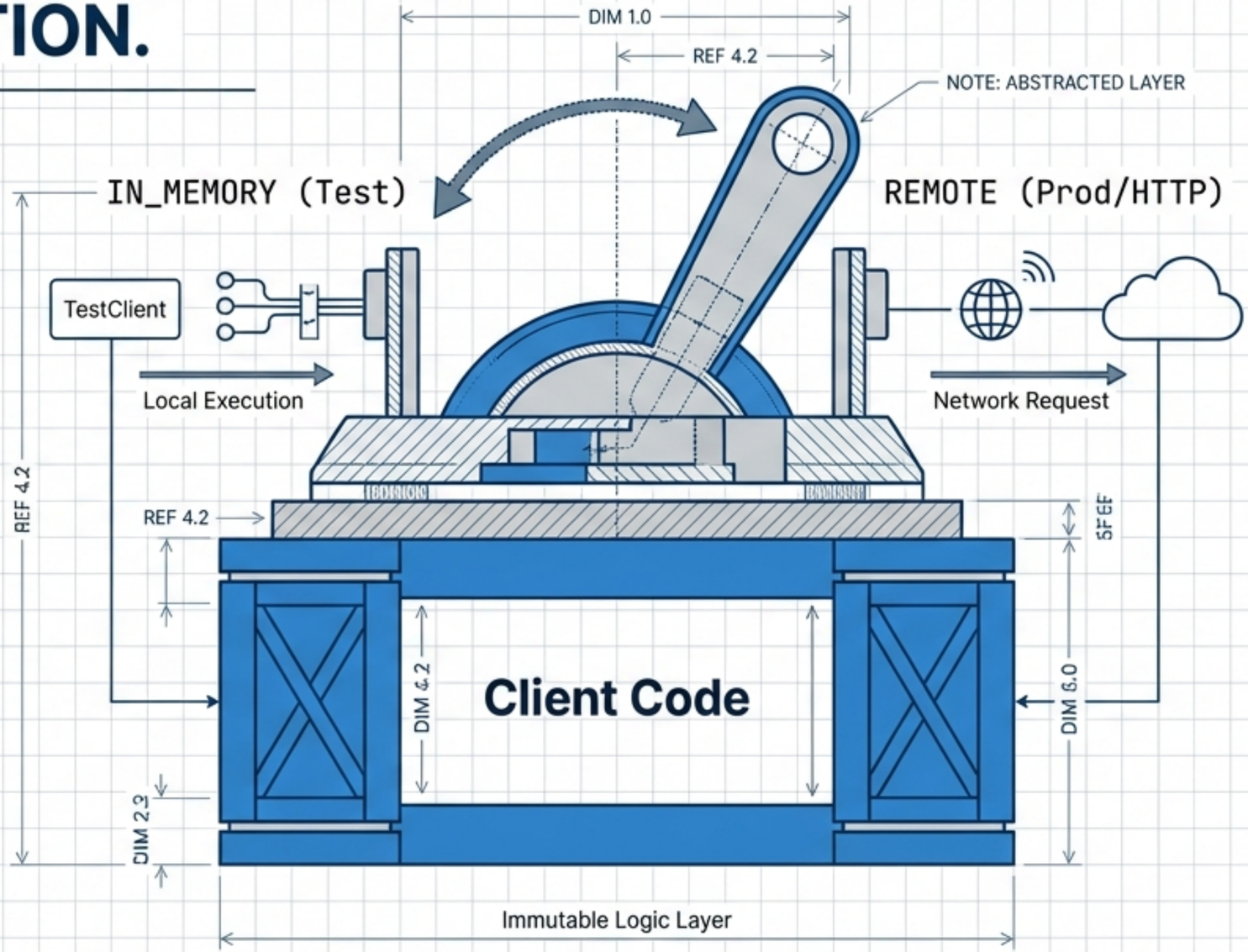
THE SHIFT:

1. **Centralisation**: Moving generic transport logic to **osbot_fast_api**.



2. **Simplification**: Eliminating wrapper classes (reducing 3 classes to 1).

3. **Standardisation**: Strictly defining domain client boundaries.



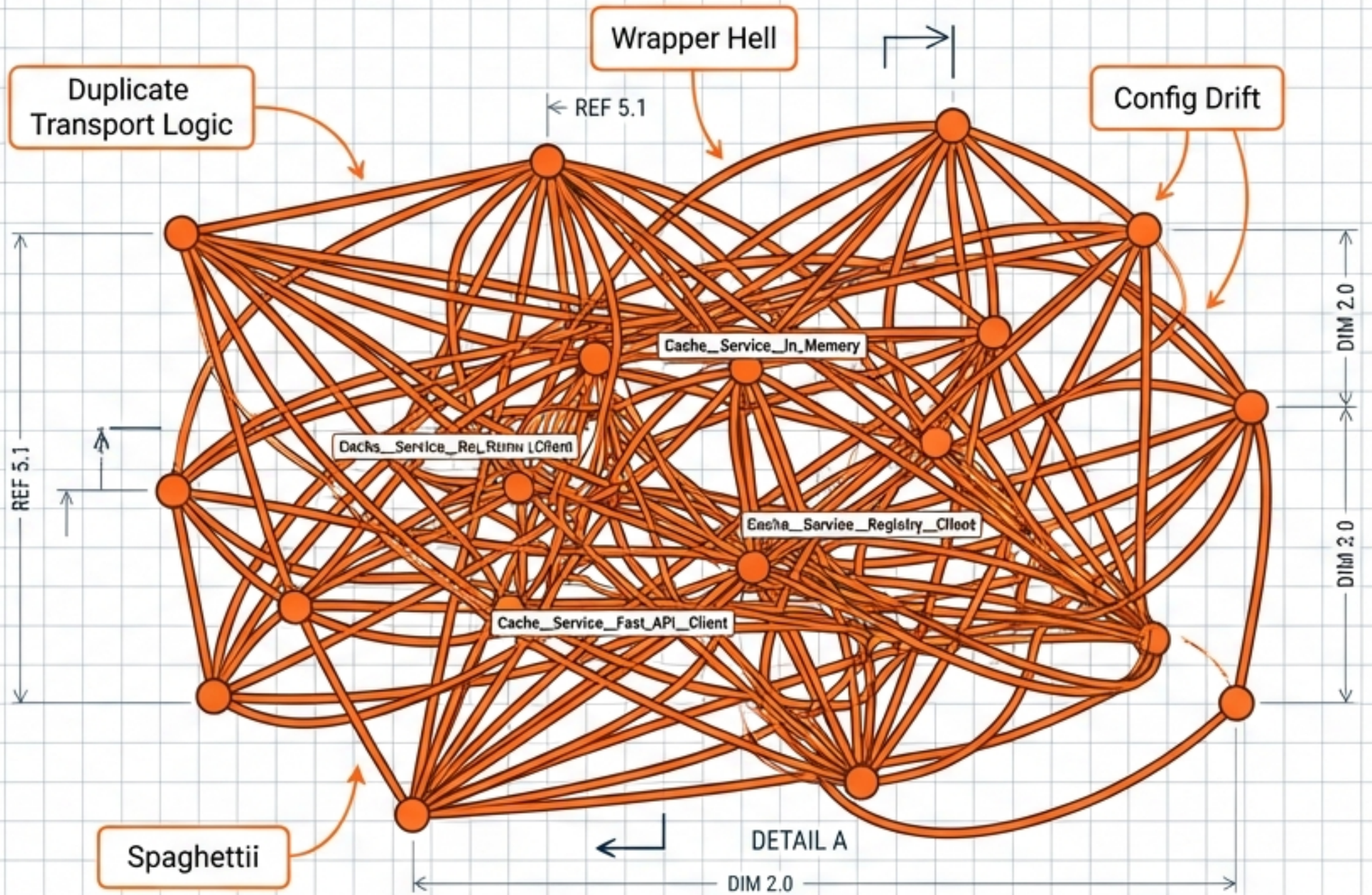
ENGINEERING SPEC - REV A

THE COST OF ORGANIC GROWTH.

As microservices multiplied, boilerplate code multiplied unchecked.

PAIN POINTS LIST

- **Duplication**: Transport logic copied into every service client.
- **Wrapper Hell**: Maintaining multiple classes for a single service.
- **Config Drift**: Configuration fields duplicated between Registry and Client.
- **Ambiguous Ownership**: Unclear who owns FastAPI app creation in IN_MEMORY mode.

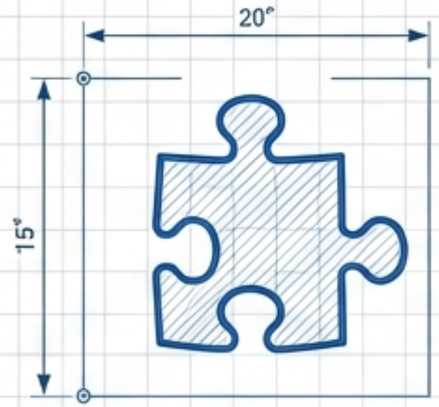


ENGINEERING SPEC - REV B

DATE

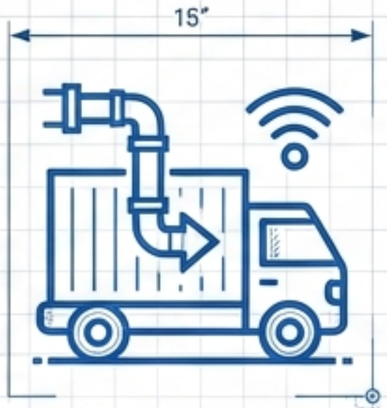
APPROVED

DECONSTRUCTING THE REDUNDANCY.



The Glue

Cache__Service__In_Memory was false abstraction.



The Transport

100% Generic. Belongs in Infra.

Identity

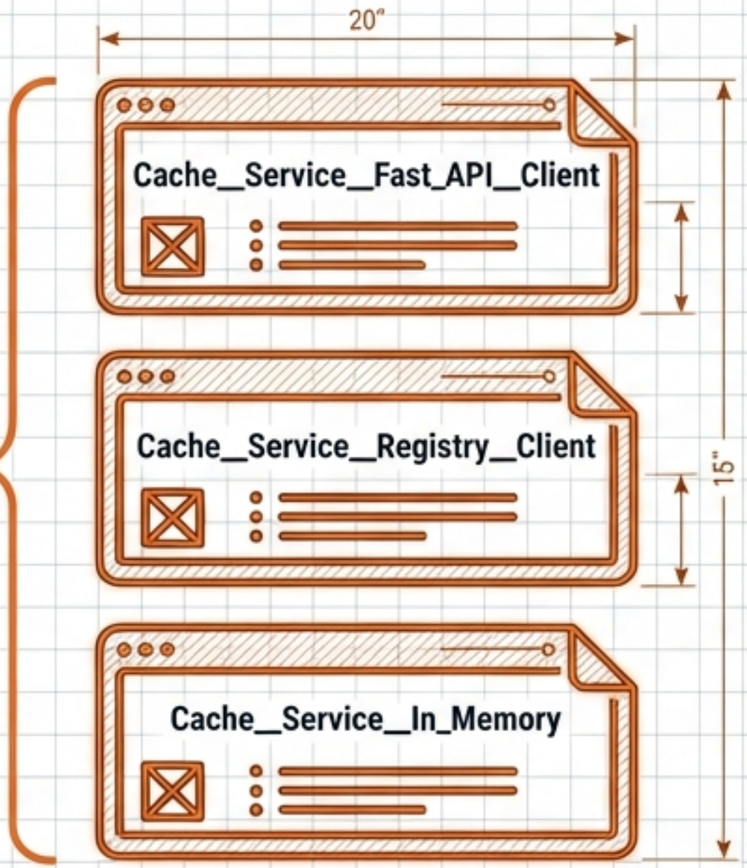
Registry is ‘How’, not ‘What’.

- **Insight 1:** “In_Memory” classes were just **bootstrapping glue.**
- **Insight 2: Transport logic is infrastructure,** not domain.
- **Insight 3:** Naming should reflect **identity,** not management method.

ENGINEERING SPEC - REV C	
Date	2022: 6 / 1
Approval	

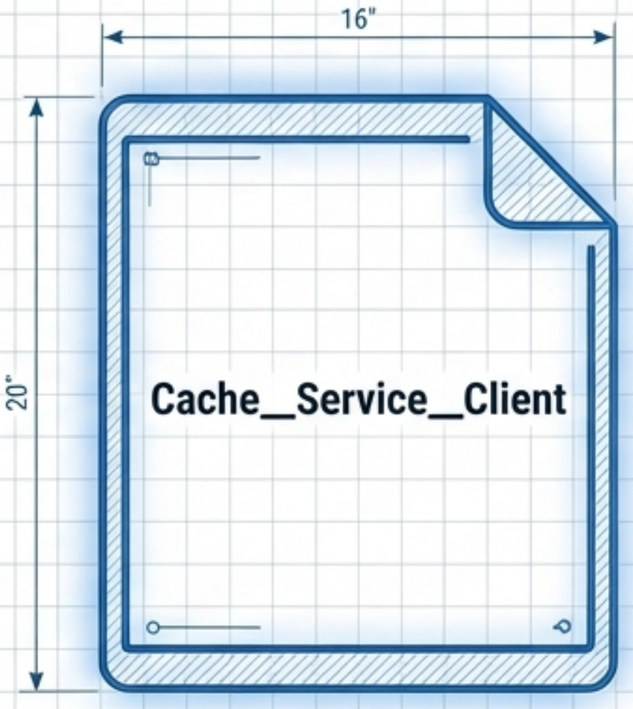
RADICALLY REDUCING COMPLEXITY.

PREVIOUS STATE



3+ classes per service.

NEW ARCHITECTURE

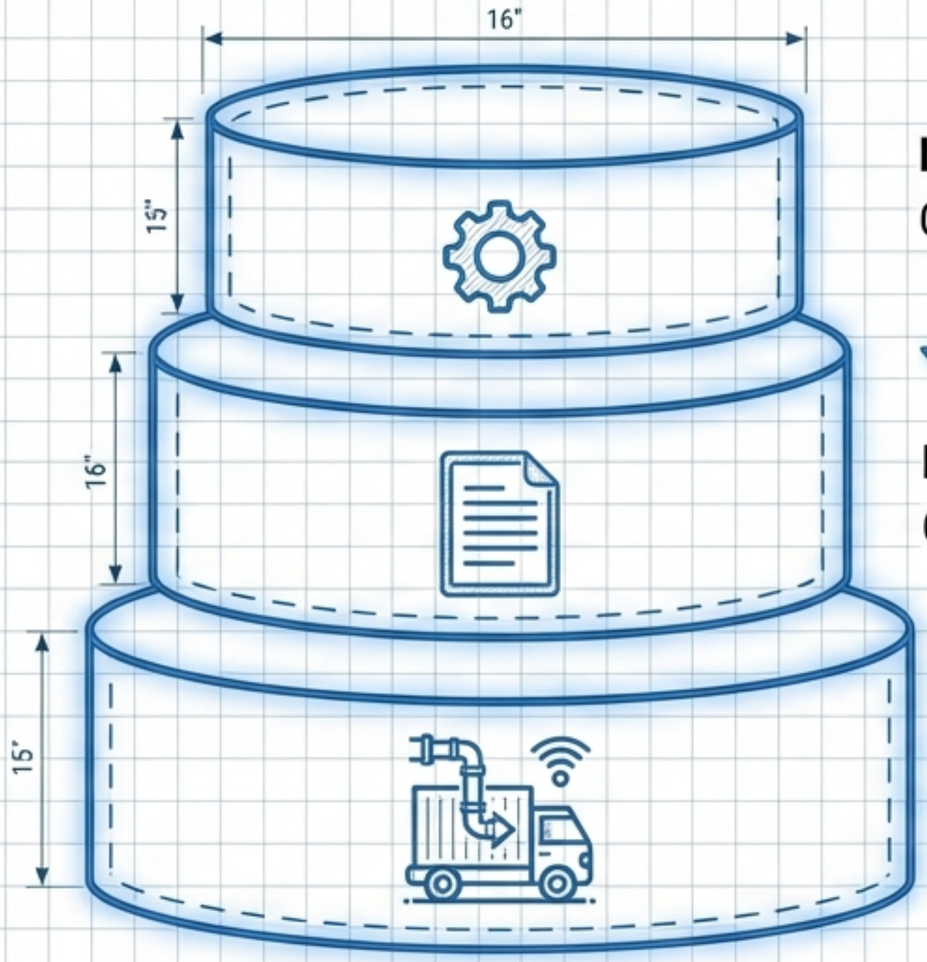


1 class per service.

Inherits:
Fast_API__Service__Registry__Client__Base

ENGINEERING SPEC - REV D	
Date	2022: 7 / 29
Approval	

CLEAR BOUNDARIES, CLEAR OWNERSHIP.



Layer 3: Service Package (mgraph_ai_service_cache)
Owns Registration Helpers (IN_MEMORY setup).

Layer 2: Domain Client Package (mgraph_ai_service_cache_client)
Owns Domain Methods (Cache__Service__Client).

Layer 1: Infrastructure (osbot_fast_api)
Owns Transport (Fast_API__Client__Requests) & Base Classes.

Infrastructure handles the ‘How’ (Transport). Domain handles the ‘What’ (Business Logic).

ENGINEERING SPEC - REV C	
Date	3022: 7 / 29
Approval	

THE ENGINE: FAST_API__CLIENT__REQUESTS

Promoting transport logic to the shared infrastructure project (osbot_fast_api).

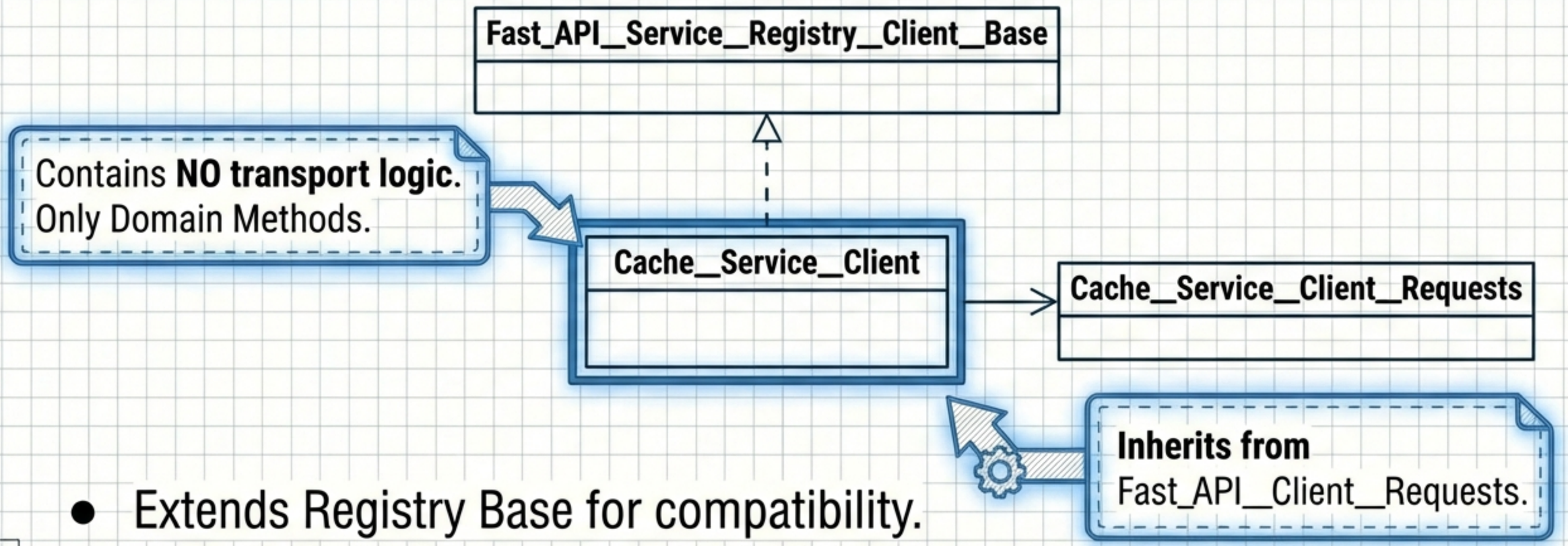
```python
class Fast_API__Client__Requests:

    def execute(self, method, path, data=None, params=None):
        # Generic HTTP execution logic
        # Handles Authentication
        # Handles Error Parsing
        # Works for ANY service
        pass
```

100% Generic.
No domain dependencies.

ENGINEERINO SPEC - REV O	
Date	3022: 7 / 29
Approval	

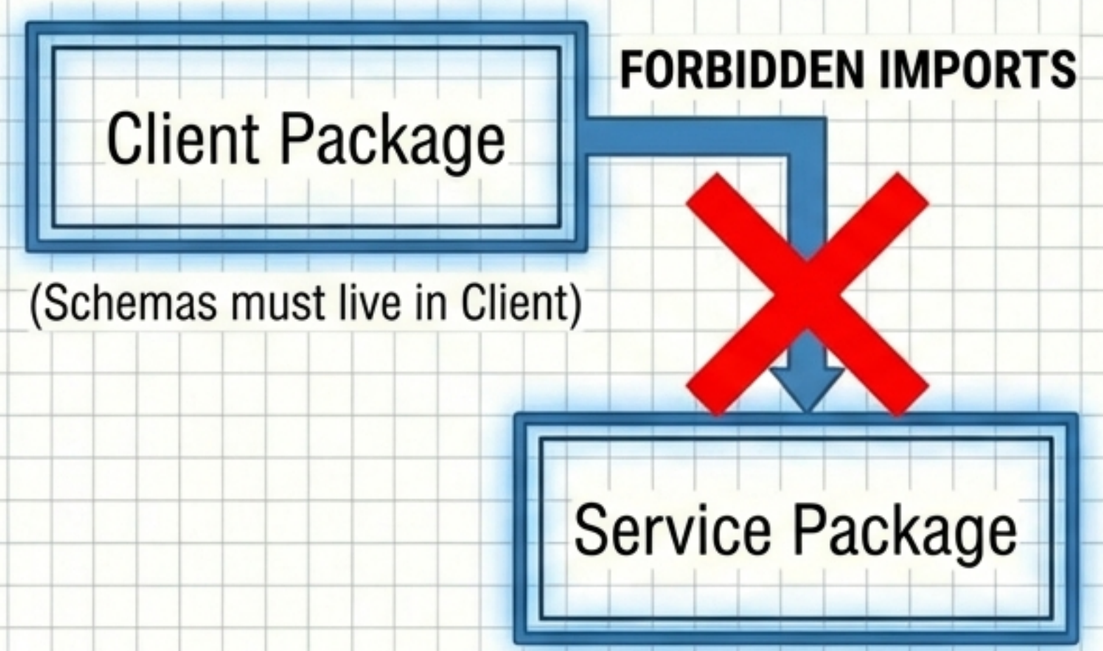
THE DOMAIN CLIENT: THIN & FOCUSED.



- Extends Registry Base for compatibility.
- Uses Client__Requests for domain extensions.
- Streamlined developer experience.

ONE-WAY DEPENDENCIES & MODE AVAILABILITY.

THE RULE



(Schemas must live in Client)

THE MATRIX

Client Only Installed	**Result: REMOTE Mode Available (HTTP)**
Client + Service Installed	**Result: REMOTE + IN_MEMORY Modes Available**

The Service acts as a plugin that enables IN_MEMORY capabilities via registration.

ONE PATTERN FOR TESTS AND PRODUCTION.

```python
# TEST SETUP
from mgraph_ai_service_cache.registration import register_cache_service

client = register_cache_service()  # Returns configured client
assert client.status() == {'status': 'ok'}
```

Identical usage

```python
# PRODUCTION SETUP
from mgraph_ai_service_cache_client import Cache__Service__Client

client = Cache__Service__Client() # Loads env vars
assert client.status() == {'status': 'ok'}
```

- No mocks needed. Tests run against real FastAPI app.

THE CLEANUP ROADMAP.

DELETE

- **Cache__Service__In_Memory.py**
- Cache__Service__**Fast_API__Client.py**
- Cache__Service__**Registry__Client.py**
- Cache__Service__**Fast_API__Client__Config.py**

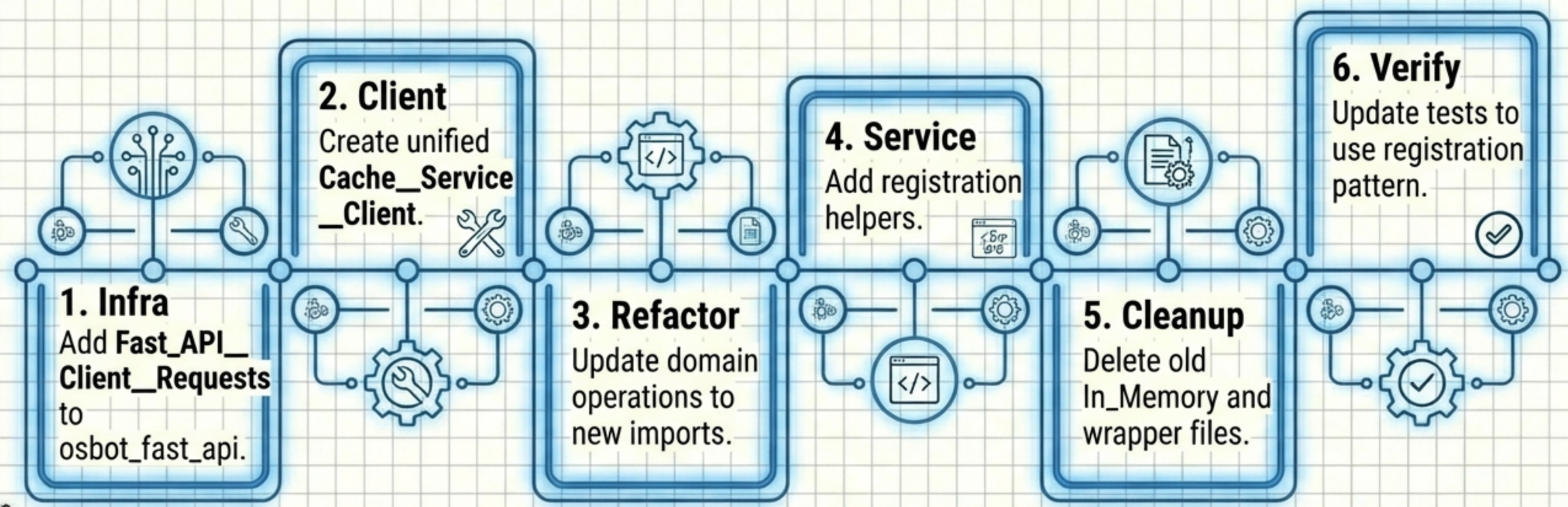
CREATE

- osbot_fast_api / **Fast_API__Client__Requests.py**
- mgraph_ai_service_cache_client **Cache__Service__Client.py**
- mgraph_ai_service_cache / registration / **register_cache_service.py**

MODIFY

- **Fast_API__Service__Registry__Client__Base.py**
- Domain Operation Files (Update Imports)

Step-by-Step Execution Order.



Why This Matters.

Complexity

Drastic reduction in file count & config schema duplication.

Speed

~100ms startup for full service stack tests.

Reliability

Same code paths for Test & Prod. No Mocks.

Extensibility

Standardised blueprint for future services (LLM, Graph).

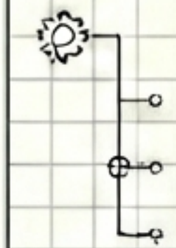
Reference: From Old to New.

Old Class Name	New Class Name
`Cache__Service__Fast_API__Client`	`Cache__Service__Client`
`Cache__Service__Registry__Client`	`Cache__Service__Client`
`Html_Graph__Service__Fast_API__Client`	`Html_Graph__Service__Client`
`LLM__Service__Fast_API__Client`	`LLM__Service__Client`

Note: Config classes merge into **Fast_API__Service__Registry__Client__Config**.

Future Considerations.

- **Result Schema**: Generic result schema vs. Raw response?
- **Error Handling**: Raise Python exceptions vs. Return error results?
- **Registry Scope**: Singleton vs. Per-application instance?
- **Async Support**: Should generic transport support async?



This architecture provides the solid foundation to answer these questions.